



JavaScript中级-对象、函数、类、异步

JS对象：

001.对象定义：

字面量定义对象：{keys: "values",,,}

创建空对象、添加属性：const x = {} ; x.keys = "values"或者const x = new Object; (避免使用new)

对象构造函数：创建对象函数，批量制造对象

```
function Person(first, last, age, eye) {
```

```
  this.firstName = first;
```

```
  this.lastName = last;
```

```
  this.age = age;
```

```
  this.eyeColor = eye;
```

```
    this.nationality = "English";赋予属性的值会成为所有对象的默认值
```

```
}
```

```
const myFather = new Person("John", "Doe", 50, "blue");
```

002.对象方法：

分类为常规方法、属性管理方法、对象保护方法

常规方法：

// 将属性从源对象复制到目标对象，理解为属性，后者给前者，同替、无补
Object.assign(target, source)

```
Object.assign(person1, person2);
```

// 从现有对象创建对象

```
Object.create(object)
```

// 返回对象的键/值对数组，理解为数组存储、便于循环、也可以转换为Map
Object.entries(object)

```

let text = Object.entries(person);
const myMap = new Map(Object.entries(fruits));
// 从键/值列表创建对象、将列表转换为对象
Object.fromEntries()
const myObj = Object.fromEntries(fruits);
// 返回对象的键数组，仅返回对象的键，作为数组
Object.keys(object)
// 返回对象的属性值数组、仅返回对象的值，作为数组
Object.values(object)
// 根据函数对对象元素进行分组，根据回调函数返回的字符串对对象元素进行分组返回对象
Object.groupBy(object, callback)
function myCallback({ quantity }) {
return quantity > 200 ? "ok" : "low";
}
// Group by Quantity
const result = Object.groupBy(fruits, myCallback);

```

属性管理方法：

```

// 添加或更改对象属性，对象、键、值，更改会是添加，看这个键存在与否
Object.defineProperty(object, property, descriptor)
Object.defineProperty(person, "year", {value:"2008"});
// 添加或更改对象属性
Object.defineProperties(object, descriptors)
Object.defineProperty(person, "language", {value : "NO"});
// 访问属性
Object.getOwnPropertyDescriptor(object, property)
// 访问属性
Object.getOwnPropertyDescriptors(object)
// 以数组形式返回所有属性
Object.getOwnPropertyNames(object)

```

Object.keys(person);可以替换

// 访问原型

Object.getPrototypeOf(object)

对象保护方法：最常见方法是使用const创建对象（对数组同样使用）

// 防止重新分配

const car = {type:"Fiat", model:"500", color:"white"};

// 防止添加对象属性、也可以对数组使用

Object.preventExtensions(object)

const fruits = ["Banana", "Orange", "Apple", "Mango"];

Object.preventExtensions(fruits);

// 如果可以向对象添加属性，则返回 true，即检查该对象是不是可以扩展的

Object.isExtensible(object)

// 防止添加和删除对象属性、冻结该对象

Object.seal(object)

// 如果对象已密封，则返回 true、检查对象是否被冻结

Object.isSealed(object)

// 防止对对象进行任何更改

Object.freeze(object)

// 如果对象已冻结，则返回 true

Object.isFrozen(object)

003.对象原型：

所有的JS对象都是从原型中继承属性与方法

使用构造函数定义对象，向构造函数添加新属性，不可直接向创建完对象添加新属性，需要找到最初构造函数（给机器加属性，不要给产品加属性）

使用prototype属性向对象构造函数添加属性：

```
function Person(first, last, age, eyecolor) {
```

```
  this.firstName = first;
```

```
  this.lastName = last;
```

```
  this.age = age;
```

```

this.eyeColor = eyecolor;
}
Person.prototype.nationality = "English"; (产品找原型，再找属性，再加值)
Person.prototype.name = function() {
return this.firstName + " " + this.lastName;
}; (产品找原型、再找方法、在构建方法)

```

004.对象访问器 (Getter、Setter)

- **Getter**：是一个函数，当访问对象的某个属性时，会自动调用这个函数，并返回一个值。它看起来像是在访问一个普通属性，但实际上执行的是一个函数。
- **Setter**：也是一个函数，当给对象的某个属性赋值时，会自动调用这个函数，并可以对赋值操作进行拦截和处理。
- 都可以使用Object.defineProperty () 方法对该对象添加Getters和Setters
-

```

Object.defineProperty(obj, "reset", {
get : function () {this.counter = 0;}
});
Object.defineProperty(obj, "subtract", {
set : function (value) {this.counter -= value;}
});

```

Getter：使用Lang属性来获取get属性的值Language

```

const person = {
firstName: "John",
lastName: "Doe",
language: "en",
get lang() {
return this.language;
}
}

```

```

};
document.getElementById("demo").innerHTML = person.lang;
Setter: 使用Lang属性来获取set的值Language
const person = {
  firstName: "John",
  lastName: "Doe",
  language: "",
  set lang(lang) {
    this.language = lang;
  }
};
person.lang = "en";
document.getElementById("demo").innerHTML = person.language;

```

JS函数:

001.函数定义: 函数声明、函数表达式

声明函数: 一般不会立即执行、保存供以后调用执行

```

function myFunction(a, b) {
  return a * b;
}

```

表达式函数: 将函数存储与变量中、该变量可以用作函数

```

const x = function (a, b) {return a * b};
let z = x(4, 3);

```

构造函数: function

函数提升: 函数可以在被声明之前调用, 表达式函数不可以

自调用函数: 函数周围添加括号、自动运行函数, 不能自行使用

```

(function () {
  let x = "Hello!!"; // I will invoke myself
})();

```

函数作值：一个函数可以调用另一个函数

```
function myFunction(a, b) {  
  return a * b;  
}
```

```
let x = myFunction(4, 3) * 2;
```

函数是对象：typeof、arguments.length 属性返回调用函数时收到的参数数量、toString()
方法以字符串形式返回函数：

```
function myFunction(a, b) {  
  return arguments.length;  
}
```

箭头函数：不需要function、return、{}，不适应对象方法、不会提升

```
const x = (x, y) => x * y;
```

002.函数参数：

函数的参数：函数**参数**是 传递给函数（并由函数接收）的实际**值**

```
function functionName(parameter1, parameter2, parameter3) {  
}
```

参数规则：参数数据类型任意、不对传递的参数类型检查、参数数量任意

默认参数：undefined、建议在函数创建时设置默认值

```
function myFunction(x, y = 10) {  
  return x + y;  
}
```

```
myFunction(5);
```

y不定义则为10

函数剩余参数：(...) 允许函数将不确定数量的参数看作数组

```
function sum(...args) {  
  let sum = 0;  
  for (let arg of args) sum += arg;  
  return sum;  
}
```

```
let x = sum(4, 9, 16, 25, 29, 100, 66, 77);
```

参数对象：参数对象包含调用（调用）函数时使用的参数数组。可以简单地使用函数来查找，arguments

```
x = findMax(1, 123, 500, 115, 44, 88);  
function findMax() {  
  let max = -Infinity;  
  for (let i = 0; i < arguments.length; i++) {  
    if (arguments[i] > max) {  
      max = arguments[i];  
    }  
  }  
  return max;  
}
```

参数通过值传递、对象通过引用传递

如果函数改变了参数的值，它不会改变参数的原始值。

对参数的改变在函数外部是不可见的（反映）。

如果函数改变对象属性，它也会改变原始值。

对对象属性的更改在函数外部可见（反映）。

003.函数调用：

直接调用、全局对象调用：

```
function myFunction(a, b) {  
  return a * b;  
}
```

myFunction(4, 5) 和 window.myFunction(4, 5) 是相同的函数

关键字this：不同情况指代不同对象，在调用一个没有所有者的对象时，this会成为全局对象windows

将函数作为方法调用：

```
const myObject = {
```

```

firstName:"John",
lastName: "Doe",
fullName: function () {
return this.firstName + " " + this.lastName;
}
}
myObject.fullName();

```

使用构造函数调用函数：如果函数前带有new关键字，则他是构造函数调用，可以理解为批量制造函数，类似于对象

```

// This is a function constructor:
function myFunction(arg1, arg2) {
this.firstName = arg1;
this.lastName = arg2;
}
// This creates a new object
const myObj = new myFunction("John", "Doe");
// This will return "John"
myObj.firstName;

```

004.函数重用call ()、apply ()：

通过使用call () 可以编写一个可以在不同对象上使用的方法

所有函数都是方法，下例指person的fullName方法在person2上使用

```

const person = {
fullName
: function() {
return this.firstName + " " + this.lastName;
}
}
const person1 = {

```



```
firstName:"John",
lastName: "Doe"}
const person2 = {
firstName:"Mary",
lastName: "Doe"}
// This will return "John Doe":
person.fullName.call(person1);
```

call () 函数有可选参数，
person1的函数方法，给person2使用，但是person2的属性不全，需要额外参数补充。

```
person1.mothod1.cal (person2, paramter1...)
```

通过apply () 方法，可以编写一个可以在不同对象上使用的方法，类似于call ()

```
const person = {
fullName: function() {
return this.firstName + " " + this.lastName;
}
}
const person1 = {
firstName: "Mary",
lastName: "Doe"
}
// This will return "Mary Doe":
person.fullName.apply(person1);
```

call() 和 apply() 之间的区别

区别在于：

该方法**单独** `call()` 接受参数。

该方法以**数组** `apply()` 形式接受参数。

如果您想使用数组而不是参数列表，`apply()` 方法非常方便。

```
person.fullName.apply(person1, ["Oslo", "Norway"]);
```

```
person.fullName.call(person1, "Oslo", "Norway");
```

数字列表找最大值：`Math.max(1,2,3);` // Will return 3

数组没有`max`方法，需使用：`Math.max.apply(null, [1,2,3]);` 第一个参数`null`无关紧要

005.函数借用：bind ()

使用`bind ()`，一个对象可以从另一个对象借用方法

`member` 对象从 `person` 对象借用了 `fullName` 方法：需要使用变量承接返回的运行结果

```
const person = {
  firstName:"John",
  lastName: "Doe",
  fullName: function () {
    return this.firstName + " " + this.lastName;
  }
}

const member = {
  firstName:"Hege",
  lastName: "Nilsen",
}

let fullName = person.fullName.bind(member);
```

006.闭包

变量属于局部范围或全局范围

可以使用闭包将全局变量变为局部变量。

闭包使得函数可以拥有“局部”变量。

在网页中，全局变量属于该页面。全局变量可以被页面中的所有脚本使用（或改变）。局部变量只能在定义它的函数内部使用。它是私有的，对其他函数和其他脚本代码隐藏。同名的全局变量和局部变量是不同的变量。修改其中一个变量不会修改另一个变量。

注意，未被声明的变量，始终是全局范围的，即使他是在函数内部创建内部函数plus（）可以访问父函数的变量：

```
function add() {  
  let counter = 0;  
  function plus() {counter += 1;}  
  plus();  
  return counter;  
}
```

闭包：

```
function myCounter() {  
  let counter = 0;  
  return function() {  
    counter++;  
    return counter;  
  };  
}
```

```
const add = myCounter();  
add();  
add();  
add();
```

现在计数器counter = 3

总结：

闭包是指在父函数关闭后，仍可以访问父作用域的函数。

闭包在历史上曾被用来：

- 创建私有变量

- 在函数调用之间保留状态
- 在 let 和 const 存在之前模拟块作用域
- 实现某些设计模式，如柯里化和记忆化

JS类：

001.类功能简介：

创建类：类是对象的模板，总是为类添加一个名为constructor的方法，理解为将输入的属性初始化为函数专门使用的this属性

构造函数方法constructor:

- 它必须具有确切的名称“构造函数”
- 创建新对象时自动执行
- 用于初始化对象属性

如果没有定义构造函数方法，JavaScript 将添加一个空的构造函数方法。

类方法：创建语法与对象方法相同

类中的语法建议使用严格模式编写

```
class Car {
  constructor(name, year) {

    this.name = name;
    this.year = year;
  }
  age(x) { return x - this.year; }
}
```

```
const myCar = new Car("Ford", 2014);
const year = new Date().getFullYear();
document.getElementById("demo").innerHTML = "My car is " +
myCar.age(year) + " years old.";
```

002.类继承：通过类继承创建的类可以使用另一个类的所有方法

继承对于代码的可重用性很有用：创建新类时可以重用现有类的属性与方法

创建一个Model类继承Car类的方法，使用super（）可以调用父级的属性与方法

```

class Car {
  constructor(brand) {
    this.carname = brand;
  }
  present() {
    return 'I have a ' + this.carname;
  }
}

class Model extends Car {
  constructor(brand, mod) {
    super(brand);
    this.model = mod;
  }
  show() {
    return this.present() + ', it is a ' + this.model;
  }
}

const myCar = new Model("Ford", "Mustang");
myCar.show();

```

类可以使用Getter和Setter方法，get和set关键字

对你的属性使用 getter 和 setter 是明智之举，特别是当你想在返回它们之前或者设置它们之前对它们进行一些特殊操作的时候

get获取新属性、set修改属性

```

class Car {
  constructor(brand) {
    this.carname = brand;
  }
  get cnam() {
    return this.carname;
  }
  set cnam(x) {
    this.carname = x;
  }
}

```

```
}  
  
const myCar = new Car("Ford");  
document.getElementById("demo").innerHTML = myCar.cnam;
```

类提升：类声明不会被提升，只有先声明了一个类，才能使用他

003.静态方法

静态方法是类本身定义的、不可以调用static静态对象的方法，只可以调用对象类上的方法

```
class Car {  
  constructor(name) {  
    this.name = name;  
  }  
  static hello() {  
    return "Hello!!";  
  }  
}  
  
const myCar = new Car("Ford");  
document.getElementById("demo").innerHTML = Car.hello();  
类调用静态函数可以，对象不可以即myCar.hello ()  
  
class Car {  
  constructor(name) {  
    this.name = name;  
  }  
  static hello(x) {  
    return "Hello " + x.name;  
  }  
}  
  
const myCar = new Car("Ford");  
document.getElementById("demo").innerHTML = Car.hello(myCar);
```

在类方法中可以使用参数即对象，可以在类方法中使用静态函数设置参数，然后调用类的方法时，可以将对象作为参数区调用

JS异步

001.回调：函数按照调用顺序执行

```
function myFirst() {  
  myDisplayer("Hello");  
}  
  
function mySecond() {  
  myDisplayer("Goodbye");  
}  
  
mySecond();  
myFirst();, 会显示hello
```

顺序控制：如下是计算的例子，可以完成计算，但是无法阻止计算器函数显示结果

```
function myDisplayer(some) {  
  document.getElementById("demo").innerHTML = some;  
}  
  
function myCalculator(num1, num2) {  
  let sum = num1 + num2;  
  myDisplayer(sum);  
}  
  
myCalculator(5, 5);
```

也可以调用两个函数，是一个变量去保存第一次计算结果

```
function myDisplayer(some) {  
  document.getElementById("demo").innerHTML = some;  
}  
  
function myCalculator(num1, num2) {  
  let sum = num1 + num2;  
  return sum;  
}
```

```
}  
let result = myCalculator(5, 5);  
myDisplayer(result);
```

但是上述两种方法都会有不便

回调是指将函数作为参数传递给另一个函数

```
function myDisplayer(some) {  
  document.getElementById("demo").innerHTML = some;  
}  
  
function myCalculator(num1, num2, myCallback) {  
  let sum = num1 + num2;  
  myCallback(sum);  
}  
  
myCalculator(5, 5, myDisplayer);
```

将第一个显示函数作为参数传递给计算并调用显示的函数

将函数作为参数传递时，不要使用括号

回调真正发挥作用的地方是在异步函数中，其中一个函数必须等待另一个函数（例如等待文件加载）

002.异步：与其他函数并行运行的函数，常用于回调函数

例子1：指定超时时候执行的回调函数：3秒后调用myfunction函数

```
setTimeout(myFunction, 3000);  
  
function myFunction() {  
  document.getElementById("demo").innerHTML = "I love You !!";  
}  
与下例相同，下例使用整个函数作为参数  
setTimeout(function() { myFunction("I love You !!!"); }, 3000);  
  
function myFunction(value) {  
  document.getElementById("demo").innerHTML = value;  
}
```

例子2：指定每个间隔执行的回调函数：每一秒调用一次myfunction函数


```

setInterval(myFunction, 1000);
function myFunction() {
  let d = new Date();
  document.getElementById("demo").innerHTML =
    d.getHours() + ":" +
    d.getMinutes() + ":" +
    d.getSeconds();
}

```

回调替代方案：异步编程，js程序可以长时间运行的任务，也可以并行其他任务，但是异步程序很难编写，大多数现代的异步方法都不适用回调，在js中异步编程使用promises解决

003.承诺promise：包含生产代码与消费代码的调用

promise语法：

```

let myPromise = new Promise(function(myResolve, myReject) {
  生产代码
  myResolve(); // when successful
  myReject(); // when error
});
消费代码
myPromise.then(
  function(value) { /* code if successful */ },
  function(error) { /* code if some error */ }
);

```

当身缠代码获得结果时候，它会调用两个回调函数之一

promise对象支持两种属性：状态和结果

状态pending待办、fulfilled完成、rejected拒绝

结果就是undefined不明确的、value正确结果、error错误对象

promise使用语法：接受两个参数，一个是成功回调、失败回调、两者均可选

```
myPromise.then(  
function(value) { /* code if successful */ },  
function(error) { /* code if some error */ }  
);
```

例子：

```
function myDisplayer(some) {  
document.getElementById("demo").innerHTML = some;  
}  
  
let myPromise = new Promise(function(myResolve, myReject) {  
let x = 0;  
// The producing code (this may take some time)  
if (x == 0) {  
myResolve("OK");  
} else {  
myReject("Error");  
}  
});  
  
myPromise.then(  
function(value) {myDisplayer(value);},  
function(error) {myDisplayer(error);}  
);
```

promise结合回调函数的示例

等待超时

```
let myPromise = new Promise(function(myResolve, myReject) {  
setTimeout(function() { myResolve("I love You !!"); }, 3000);  
});  
  
myPromise.then(function(value) {  
document.getElementById("demo").innerHTML = value;
```

```
});
```

等待文件加载

```
let myPromise = new Promise(function(myResolve, myReject) {  
  let req = new XMLHttpRequest();  
  req.open('GET', "mycar.html");  
  req.onload = function() {  
    if (req.status == 200) {  
      myResolve(req.response);  
    } else {  
      myReject("File not Found");  
    }  
  };  
  req.send();  
});  
myPromise.then(  
  function(value) {myDisplayer(value);},  
  function(error) {myDisplayer(error);}  
);
```

004.async、await使得promise更简便编写，即异步和等待

异步语法async：使得函数返回一个promise

```
async function myFunction() {  
  return "Hello";  
}等同于直接返回promise  
function myFunction() {  
  return Promise.resolve("Hello");  
}
```

promise语法：

```
myFunction().then(  
  function(value) { /* code if successful */ },
```

```
function(error) { /* code if some error */ }  
);
```

异步与promise:

```
async function myFunction() {  
  return "Hello";  
}  
  
myFunction().then(  
  function(value) {myDisplayer(value);},  
  function(error) {myDisplayer(error);} (可以删除错误，这样就仅会返回正常值)  
);
```

等待语法await: 使函数暂停执行，等待已解决的promise之后再继续

```
let value = await promise;
```

语法:

```
async function myDisplay() {  
  let myPromise = new Promise(function(resolve, reject) {(reject也可以省略)  
    resolve("I love You !!"); (很多时候不需要拒绝失败的选项)  
  });  
  document.getElementById("demo").innerHTML = await myPromise;  
}  
  
myDisplay();
```

等待超时:

```
async function myDisplay() {  
  let myPromise = new Promise(function(resolve) {  
    setTimeout(function() {resolve("I love You !!");}, 3000);  
  });  
  document.getElementById("demo").innerHTML = await myPromise;  
}  
  
myDisplay();
```

等待文件:

```
async function getFile() {  
  let myPromise = new Promise(function(resolve) {  
    let req = new XMLHttpRequest();  
    req.open('GET', "mycar.html");  
    req.onload = function() {  
      if (req.status == 200) {  
        resolve(req.response);  
      } else {  
        resolve("File not Found");  
      }  
    };  
    req.send();  
  });  
  document.getElementById("demo").innerHTML = await myPromise;  
}  
getFile();
```